

CONTAINERS E DOCKER

Build multi-stage

Do binário em C à aplicação React + Flask
com Docker Compose e validação 5C

Aula da semana 4



Objetivos da aula

A turma acompanha uma mesma ideia em três escalas de complexidade

01

**Separar
ambiente de build e
execução**

02

**Explicar o artefato que
atravessa os estágios**

03

**Distinguir Dockerfile,
Compose, porta e rede**

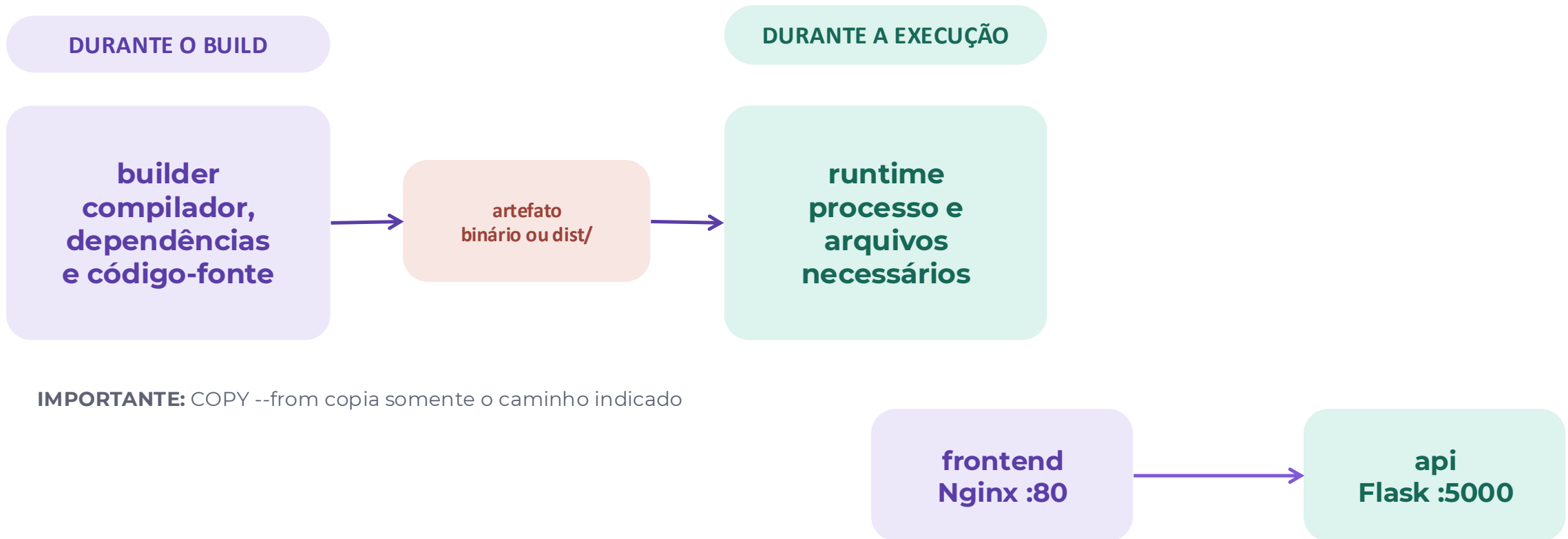
04

**Construir e comprovar
uma solução no ciclo 5C**

A demonstração começa com um binário pequeno e termina em uma arquitetura web com dois serviços.

Build e execução são fluxos distintos

O Dockerfile prepara a imagem. O Compose organiza containers já construídos.



Rede Compose: Rede no Docker Compose é uma rede virtual criada automaticamente (ou definida manualmente no arquivo docker-compose.yml) que permite que os containers de um mesmo projeto se comuniquem entre si pelo nome do serviço, isolados de outros projetos, sem precisar expor portas para o host.

Demonstração 1: Hello World em C

O compilador aparece no estágio de build e desaparece da imagem final.

```
FROM alpine:3.20 AS builder
RUN apk add --no-cache build-base
WORKDIR /src
COPY hello.c .
RUN gcc -static -Os -s -o hello hello.c
```

```
FROM scratch AS runtime
COPY --from=builder /src/hello /hello
ENTRYPOINT ["/hello"]
```

O que fica em cada estágio

builder
Alpine + gcc + headers + fonte C

runtime
Somente /hello dentro de scratch

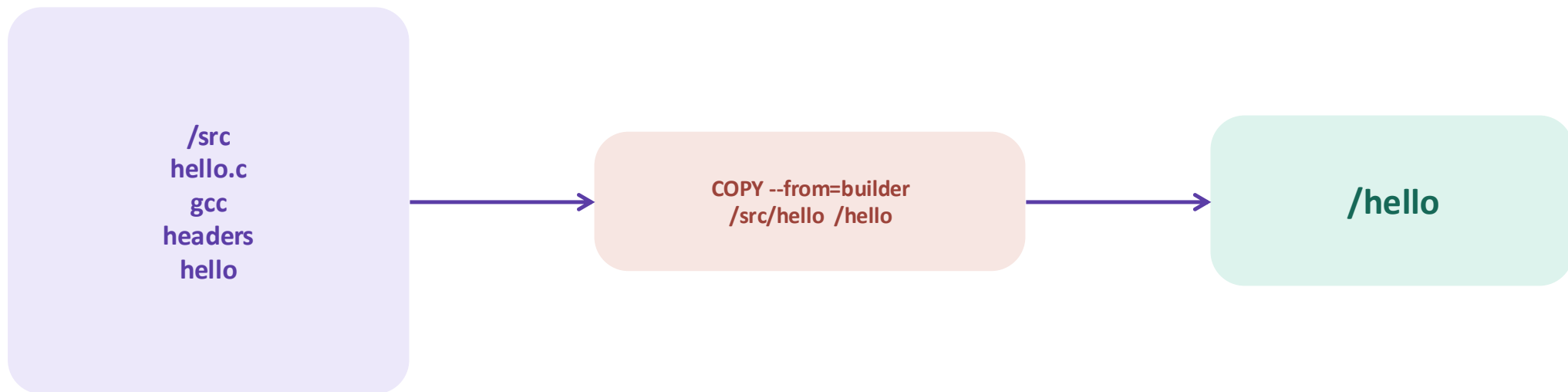
Pergunta para a turma: por que a imagem final não precisa de gcc?

Para Rodar:

```
docker build -t hello-c-multistage:1.0 .
docker run --rm hello-c-multistage:1.0
```

A fronteira entre os estágios

O estágio final recebe o artefato escolhido. Ele não herda automaticamente a imagem anterior.



O caminho copiado define o conteúdo da imagem final.

Compilador, fonte e arquivos temporários ficam no estágio de build, salvo se você os copiar explicitamente.

Demonstração 2: calculadora web

React no navegador, Nginx no frontend e Flask na API.



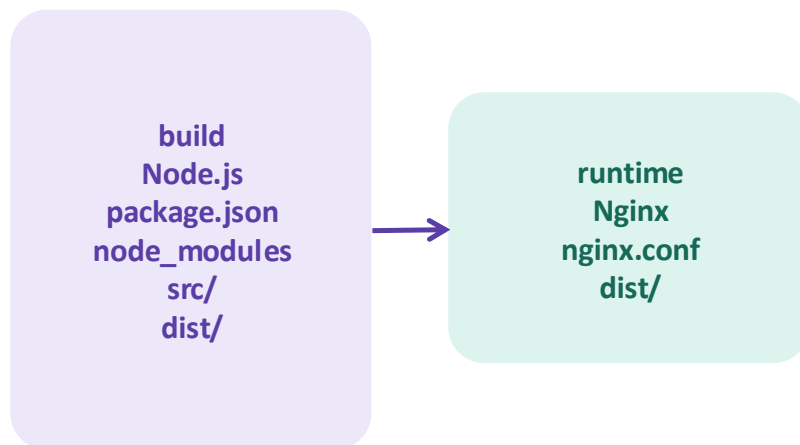
O Nginx encaminha `/api/calculador` para `http://api:5000/calculador` dentro da rede do Compose.

Frontend React: Node só participa do build

O Nginx recebe arquivos estáticos. Ele não precisa de node_modules ou do compilador do Vite.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:1.27-alpine AS runtime
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
```



artefato: /app/dist

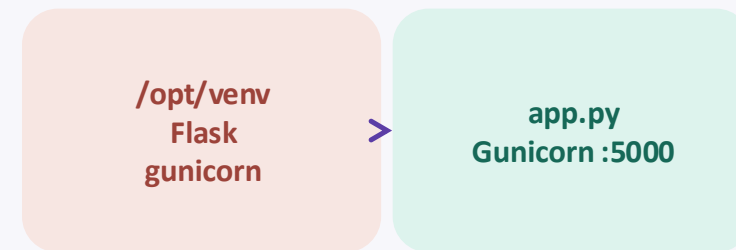
A ordem de COPY também ajuda o cache: dependências só são reinstaladas quando package.json muda.

API Flask: dependências preparadas antes

O ambiente virtual é montado no builder e copiado para a imagem de execução.

```
FROM python:3.12-slim AS builder
ENV VIRTUAL_ENV=/opt/venv
RUN python -m venv "$VIRTUAL_ENV"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
COPY --from=builder /opt/venv /opt/venv
COPY app.py .
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```



O estágio final recebe o ambiente virtual e o código que precisa executar. As ferramentas usadas no preparo não precisam estar nele.

EXPOSE 5000 documenta a porta da imagem. A publicação da porta continua sendo uma decisão do Compose.

Dockerfile e Compose têm responsabilidades diferentes

A separação evita um erro recorrente: tentar declarar estágios de build no Compose.

Dockerfile

Define a imagem

FROM e estágios
RUN e COPY
COPY --from
CMD e EXPOSE

compose.yaml

Cria e conecta serviços

services
build e ports
networks
depends_on

PARA GUARDAR:
Multi-stage fica no Dockerfile. Rede e portas ficam no Compose.

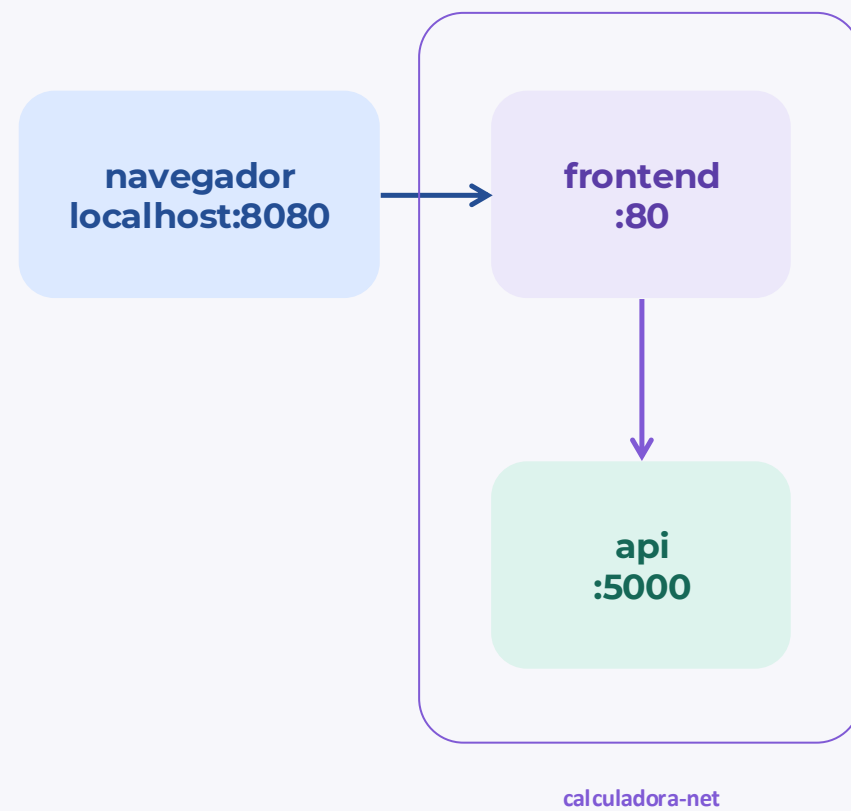
Compose: rede interna e porta publicada

O frontend atende o navegador. A API permanece acessível apenas na rede dos serviços.

```
services:
  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    networks: [calculadora-net]

  api:
    build: ./api
    expose: ["5000"]
    networks: [calculadora-net]

networks:
  calculadora-net:
    driver: bridge
```



ports publica uma porta no computador. expose documenta a porta interna do serviço.

Demonstração ao vivo

O teste precisa revelar tanto a interface quanto o estado dos serviços.

01
config

02
build e up

03
calcular

04
inspecionar

`docker compose config` valida e exibe a configuração final "achatada" do projeto, com todos os arquivos, overrides e variáveis já combinados e resolvidos.

```
docker compose config
docker compose up --build -d
docker compose ps
docker compose logs -f
docker compose down
```

`docker compose ps` lista os containers do projeto atual (definidos no `docker-compose.yml`), mostrando status, portas e serviço a que pertencem.

Evidência esperada: a calculadora abre em `localhost:8080`, a operação retorna um resultado e a API não possui porta publicada no computador.

Desafio 5C: backend em Go

A interface React permanece a mesma. A equipe reconstrói a infraestrutura da nova API.

**API Go
fornecida**

**Dockerfile
multi-stage**

**compose.yaml
com rede**

**evidência
no registro 5C**

Contrato que precisa continuar funcionando

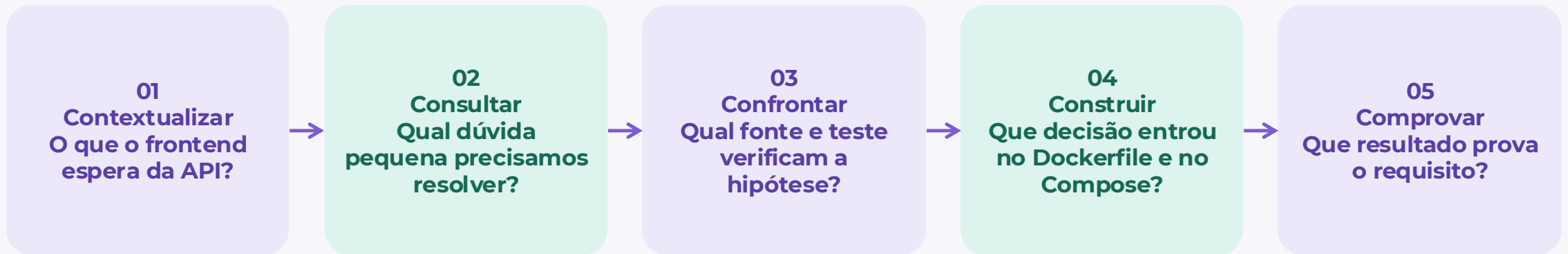
```
POST /calcular
{ "a": 10, "b": 2, "operacao": "dividir" }
```

```
200 { "resultado": 5 }
```

O frontend não aparece nesta pasta do desafio.
A equipe aponta o contexto de build para o
frontend já apresentado na demonstração
anterior.

O ciclo 5C no desafio

A resposta de uma IA ou de um colega entra como hipótese e precisa de evidência.



A entrega combina arquivo criado, execução observável e explicação da decisão tomada.

Critérios de domínio

Ao final, cada estudante deve conseguir explicar estas quatro relações.

**Estágio de build
produz um artefato**

**COPY --from
seleciona o que passa**

**Dockerfile
constrói a imagem**

**Compose
conecta os containers**

A evidência final não é apenas o container em execução. É a capacidade de justificar o que foi construído, copiado e conectado.